Thread MeshCoP Commissioning Guide

Contents

1. Native OTBR (Raspberry Pi)
2. Build Python Controller
3. Build chip-tool
4. ESP-IDF Setup (One-Time)
5. Build and Flash ESP32-H2 Firmware
6. Changing the Discriminator (12-bit vs 4-bit test cases)
7. Get the Pi's IPv6 Address
8. Get the Thread Border Agent Port
9. Run Python Test
10. Commissioning Commands
11. Troubleshooting

---

1. Native OTBR (Raspberry Pi)

Setup

```
git clone https://github.com/openthread/ot-br-posix.git ~/ot-br-posix
cd ~/ot-br-posix
sudo ./script/bootstrap
INFRA_IF_NAME=wlan0 ./script/setup
```

The `otbr-agent` service is now enabled and will start upon reboot. To instead start the service immediately without rebooting, use the `server` script:

```
./script/server
```

Config: `/etc/default/otbr-agent`

```
OTBR_AGENT_OPTS="-I wpan0 -B wlan0 spinel+hdlc+uart:///dev/ttyACM0
trel://wlan0"
OTBR_NO_AUTO_ATTACH=0
```

```
sudo systemctl restart otbr-agent
```

Web UI on port 8080

```
sudo systemctl edit otbr-web
```

Add:

```
[Service]
Environment="OTBR_WEB_OPTS=-I wpan0 -a 0.0.0.0 -p 8080"
```

```
sudo systemctl restart otbr-web
```

Access at: `http://<pi-ip>:8080/`

Service status

```
sudo systemctl status otbr-agent
sudo systemctl status otbr-web
sudo journalctl -u otbr-web -n 50 --no-pager
```

Core commands

```
sudo ot-ctl state                          # leader / router / child
sudo ot-ctl ifconfig up
sudo ot-ctl thread start
sudo ot-ctl dataset active -x              # get dataset hex
sudo ot-ctl dataset set active <hex>       # set dataset
sudo ot-ctl ba port                        # Border Agent port
sudo ot-ctl scan
sudo ot-ctl factoryreset
```

---

2. Build Python Controller

```
cd ~/connectedhomeip
python3 scripts/checkout_submodules.py --platform linux --shallow --
recursive
source scripts/bootstrap.sh
source scripts/activate.sh
./scripts/build_python.sh -i out/python_env --enable_thread_meshcop true
```

---

3. Build chip-tool

```
cd ~/connectedhomeip
scripts/examples/gn_build_example.sh examples/chip-tool out/chip-tool
'chip_mdns="platform"'
```

---

4. ESP-IDF Setup (One-Time)

> **Note:** Thread MeshCoP is not supported in ESP-IDF v5.5.1. Use the latest available tag instead — currently **v5.5.5**.

```
cd ~
git clone -b v5.5.5 --recursive --depth 1 --shallow-submodule https://
github.com/espressif/esp-idf.git
cd esp-idf
./install.sh
```

To update an existing ESP-IDF checkout to v5.5.5:

```
cd path/to/esp-idf
git fetch --depth 1 origin v5.5.5
git reset --hard FETCH_HEAD
git submodule update --depth 1 --recursive --init
git clean -ffdx
./install.sh
```

Matter environment for ESP32

```
cd ~/connectedhomeip
source scripts/bootstrap.sh -p all,esp32
```

---

5. Build and Flash ESP32-H2 Firmware

Every session, activate both environments in this order:

```
cd path/to/esp-idf
source export.sh

cd ~/connectedhomeip
source scripts/activate.sh

export IDF_CCACHE_ENABLE=1   # optional
```

Build

```
cd examples/all-clusters-app/esp32
idf.py set-target esp32h2
idf.py -D 'SDKCONFIG_DEFAULTS=sdkconfig_m5stack.defaults' build
```

> The `SDKCONFIG_DEFAULTS` file name changes depending on the target chip — e.g. use the defaults file matching your board (H2, C3, C6, etc.). Check the `examples/all-clusters-app/esp32` directory for the available `sdkconfig_*.defaults` files and pick the one matching your target.

Clean rebuild if switching chip targets:

```
rm -rf build sdkconfig managed_components dependencies.lock
idf.py set-target esp32h2
idf.py -D 'SDKCONFIG_DEFAULTS=sdkconfig_m5stack.defaults' build
```

Flash & monitor

Every session, activate both environments (esp-idf `export.sh` then Matter `scripts/activate.sh`) in the **same shell** before running `idf.py` — see step 5 above. Skipping this is the most common cause of build/flash failures.

```
idf.py -p /dev/ttyUSB0 erase-flash
idf.py -p /dev/ttyUSB0 flash monitor
```

Exit monitor: `Ctrl+]`

> **Always factory-reset the device before a test run — a reboot or reset-button press is not enough.** A hardware reset (or the board's `EN` /RESET button) only reboots the CPU; it does not touch NVS, so a previously-commissioned board will boot with its old Matter fabric and Thread dataset fully intact and will not appear "factory-fresh" to the test. You have two ways to actually clear this state:
>
> **Option A — UART factory-reset command (fast, no reflash needed):**
>
> ```
> sudo minicom -D /dev/ttyUSB0 -b 115200
> ```
>
> Once connected to the device console, type:
>
> ```
> matter device factoryreset
> ```
>
> The device will clear its Matter fabric and Thread dataset and reboot on its own. This is the quickest way to reset between repeated commissioning attempts on firmware that's already flashed — use this by default.
>
> **Option B — Full erase-flash (use after building new firmware, or if Option A is unavailable):**
>
> ```
> idf.py -p /dev/ttyUSB0 erase-flash
> idf.py -p /dev/ttyUSB0 flash monitor
> ```
>
> **Either way, confirm it actually worked** by checking the boot log does **not** contain a line like `Fabric index 0x1 was retrieved from storage` — if it does, the reset didn't take effect and needs to be re-run.

---

6. Changing the Discriminator (12-bit vs 4-bit test cases)

The device has a **12-bit long discriminator**. The **short (4-bit) discriminator** is just its top 4 bits: `short = long >> 8`.

- A **QR code** carries the full long discriminator, so commissioning with a QR code always uses the long path.
- A **manual pairing code** carries only the short discriminator, so commissioning with a manual code uses the short path.
- To get a short discriminator of `N`, set the long discriminator to `N << 8` (for example short `0xA` needs long `0xA00`, **not** `0xA`).

Long discriminator	Short	QR code (passcode 20202021)	Manual pairing code
`10` (`0x00A`)	`0x0`	`MT:-24J0YX852ZZ.548G00`	`00054912336`
`2560` (`0xA00`)	`0xA`	`MT:-24J0I9U40C60648G00`	`23331712339`
`3840` (`0xF00`, default)	`0xF`	`MT:-24J0I9U40KA0648G00`	`34970112332`
`3857` (`0xF11`)	`0xF`	`MT:-24J0QL051MA0648G00`	`34970112332`

All codes above use VID `0xFFF1`, PID `0x8001`, passcode `20202021` and the Thread rendezvous capability. Note that `0xF00` and `0xF11` share a manual code, because they have the same short discriminator.

Option 1 (easy, recommended): change it at runtime from the device shell

No rebuild or reflash needed.

1. Open the device console:

```
sudo minicom -D /dev/ttyUSB0 -b 115200
```

Exit minicom with `Ctrl+A`, then `X`.

2. Set the discriminator (decimal value, 12-bit):

```
matter config discriminator 10
```

3. Confirm:

```
matter config
```

Expected output (the discriminator is shown in hex):

```
VendorId:        65521 (0xFFF1)
ProductId:       32769 (0x8001)
HardwareVersion: 0 (0x0)
PinCode:         20202021
Discriminator:   00a
Done
```

4. Use the matching code from the table above. The QR and manual codes printed in the boot log may still show the old values, so do not rely on them after a runtime change.

> **Always re-run `matter config` before a test run**, especially after a reboot, factory reset or reflash. If the discriminator is back to the default, set it again.

Option 2 (advanced): set it at build time

Use this if the runtime command is not available on your firmware.

1. Create `main/CHIPProjectConfig.h`:

```
cd ~/master/connectedhomeip/examples/all-clusters-app/esp32
nano main/CHIPProjectConfig.h
```

Add:

```
#pragma once
#define CHIP_DEVICE_CONFIG_USE_TEST_SETUP_DISCRIMINATOR 0xF11
```

Save: `Ctrl+O`, Enter, then exit: `Ctrl+X`.

The value is the **12-bit long discriminator**. `0xF11` is a 12-bit test case (short `0xF`); for a 4-bit test case use `short << 8`, e.g. `0xA00` for short `0xA`.

2. Add this line to `sdkconfig.defaults.esp32h2.tc` (one-time):

```
nano sdkconfig.defaults.esp32h2.tc
```

Go to the end of the file and add:

```
CONFIG_CHIP_PROJECT_CONFIG="main/CHIPProjectConfig.h"
```

Save: `Ctrl+O`, Enter, then exit: `Ctrl+X`.

3. Rebuild and reflash:

```
idf.py -D 'SDKCONFIG_DEFAULTS=sdkconfig.defaults.esp32h2.tc' build
idf.py -p /dev/ttyUSB0 erase-flash
idf.py -p /dev/ttyUSB0 flash monitor
```

4. Confirm: manual pairing code / QR code in boot log should differ from the default (`34970112332` / `MT:-24J0I9U40KA0648G00`).

To switch discriminators, edit `CHIPProjectConfig.h` and repeat steps 1, 3, 4.

---

7. Get the Pi's IPv6 Address (for `--thread-ba-host`)

```
ip -6 addr show wlan0
```

This lists several addresses. Use the one marked `mngtmpaddr` — this is the stable address. **Do not use** the one marked `temporary` — it rotates every ~30 minutes and will stop working mid-session.

Example output:

```
inet6 fd24:add3:5350:49e1:xxxx:xxxx:xxxx:xxxx scope global temporary
dynamic        <- do NOT use
inet6 fd24:add3:5350:49e1:yyyy:yyyy:yyyy:yyyy scope global dynamic
mngtmpaddr ...    <- use this one
inet6 fe80::zzzz:zzzz:zzzz:zzzz scope
link                                        <- link-local, skip
```

Use the `mngtmpaddr` address as `<pi-ip>` in the commands below. If your Pi is on Ethernet instead of Wi-Fi, run the same command against `eth0` instead of `wlan0`.

---

8. Get the Thread Border Agent Port (for `--thread-ba-port`)

```
sudo ot-ctl ba port
```

This returns the current MeshCoP Border Agent port (commonly `49154`, but not guaranteed — it can change across reboots/dataset changes, so always re-check rather than assuming).

> Returns `0` if Thread hasn't formed/joined a network yet. Run `sudo ot-ctl state` first — it should report `leader` (or `router` / `child`) before checking the port. If it's still `0` after that, run `ifconfig up` and `thread start` (see Section 1, Core commands) and check again.

---

9. Run Python Test

```
python3 TC_SC_TC_2_1.py \
  --in-test-commissioning-method thread-meshcop \
  --thread-dataset-hex <dataset-hex> \
  --thread-ba-host <pi-ip> \
  --thread-ba-port <port> \
  --discriminator 3840 \
  --passcode 20202021 \
  --no-wildcard-subscription
```

`--discriminator` must equal the DUT's long (12-bit) discriminator. Check it with `matter config` on the device console (Section 6).

---

10. Commissioning Commands

```
# Thread MeshCoP
./out/chip-tool/chip-tool pairing thread-meshcop <node-id> \
  hex:<dataset-hex> <manual-pairing-code> \
  --thread-ba-host <pi-ip> --thread-ba-port <port>

# (use a QR code, e.g. MT:-24J0YX852ZZ.548G00, in place of the manual code to take the
#  long-discriminator path; a manual code takes the short-discriminator path. See Section 6)

# BLE-then-Thread (simpler, uses discriminator + passcode)
./out/chip-tool/chip-tool pairing ble-thread <node-id> \
  hex:<dataset-hex> 20202021 3840
```

Troubleshooting

Symptom	Fix
`otbr-web` crash-loop, "Address already in use (errno 98)" on port 80/8080	`docker ps -a` → stop/remove whatever's holding the port: `docker stop otbr && docker rm otbr`
`ot-ctl ba port` returns `0`	Thread not formed yet — check `ot-ctl state`, run `ifconfig up` + `thread start`
Config edits not applying	`sudo systemctl restart otbr-agent` after editing `/etc/default/otbr-agent`
Petition/commissioning fails instantly (<50ms)	Port/network reachability issue — check `-B` backbone interface matches real LAN interface
Intermittent mDNS SRV parse failures during Thread MeshCoP	Use **native** OTBR, not Docker — Avahi reflects mDNS across all Docker bridges/veths and duplicates records, causing parse races
ESP32 build fails after switching chip targets	`rm -rf build sdkconfig managed_components dependencies.lock`, re-run `set-target` + `build`
ESP32 not flashing / not detected	Confirm correct `/dev/ttyACM0` or `/dev/ttyUSB0` port; hold BOOT button during flash if required by the board

Symptom	Fix
`chip-tool` "Invalid address" error on `--thread-ba-host`	Binary may be built with `chip_inet_config_enable_ipv4=false` — use an IPv6 address instead, or rebuild with IPv4 enabled
Thread MeshCoP not working on ESP-IDF	Confirm you're on v5.5.5 or later, not v5.5.1
DUT jumps straight to `leader` / `router` / logs "Fabric index... retrieved from storage" on boot despite reflashing	Device is still commissioned from a previous run — a reboot or reset-button press does NOT clear this. Run `matter device factoryreset` via UART (`sudo minicom -D /dev/ttyUSB0 -b 115200`), or do a full `idf.py -p <port> erase-flash`
Pressing the board's reset/EN button doesn't un-commission the device	Expected — the reset button only reboots the CPU, it never touches NVS. Use `matter device factoryreset` via UART or `erase-flash` instead
`config discriminator` via UART fails with `Error: <number>`	Use the full command with the `matter` prefix: `matter config discriminator <value>` (Section 6, Option 1). If it still fails, use the build-time method (Section 6, Option 2)
`CONFIG_CHIP_...` line added to `sdkconfig.defaults.*` has no effect	The discriminator macro is not a Kconfig symbol — it can't be set directly in a defaults file. It must go in `CHIPProjectConfig.h` ; the defaults file only needs the one `CONFIG_CHIP_PROJECT_CONFIG="main/CHIPProjectConfig.h"` line pointing to it
`idf.py menuconfig` arrow keys print `^[OB` / navigation doesn't work over SSH	Terminal escape-sequence mismatch — skip menuconfig, set values directly via `idf.py -D CONFIG_X=Y build` , or add the line manually to the sdkconfig defaults file with a text editor
`idf.py erase-flash` / `build` fails with `ModuleNotFoundError: No module named 'python_path'` / CMake `chip_codegen.cmake` error	Matter environment not sourced in this shell — run `source esp-idf/export.sh` then `source connectedhomeip/scripts/activate.sh` , in that order, in the same terminal, before running `idf.py`
`esptool` / `idf.py` fails with "device reports readiness to read but returned no data (device disconnected or multiple access on port?)"	Another process (commonly a leftover `minicom` / `screen` session) still has the serial port open — check with `sudo lsof /dev/ttyUSB0` and kill/close it, then retry

Symptom	Fix
Short-discriminator test uses the wrong value (e.g. set `10` expecting short `0xA`)	The setting takes the 12-bit long value; `short = long >> 8`. For short `0xA` set long `0xA00` (2560). See the table in Section 6